

Report PacMan

Vico Aerts (s0230569)

1 Introduction

This report details the design and implementation of an interactive game inspired by Pac-Man, developed in C++ using the SFML graphics library. The primary focus of this project is not only to recreate the core gameplay mechanics, but to demonstrate a well-designed architecture following design patterns and utilize advanced C++ features.

2 Gameplay features

The game recreates classic Pac-Man gameplay in which the player navigates a maze, collects items, avoids ghosts and progresses through multiple levels.

2.1 Gameplay and Entities

Pac-Man moves continuously through the maze and starts with three lives. Collisions with ghosts result in a life loss and a reset of entity positions. The game ends when no lives remain. Top 5 scores will be saved.

Four ghosts are present, each with a chasing behavior based on Manhattan distance, random movement or Pac-Man's facing direction(tries to cut pacman off).

2.2 Items and Scoring

Each second, 1 point gets deducted

- Coins: +10 points, with a time-based bonus for rapid collection.
- Fruits: +50 points and activates ghost fear mode.
- Ghosts (fear mode): +200 points when eaten with additional chaining bonus.
- Level completion: 500 bonus points awarded after collecting all items.

Scores persist across levels and are stored in a top-five high score list.

2.3 Difficulty Scaling

With each new level:

- Ghost speed increases 2% per level (capped at lvl 20).
- Fear mode duration decreases -0.3s each level (capped at lvl 20).

3 Architecture and Design Patterns

The project employs several design patterns to ensure a flexible, extendable, and modular codebase.

3.1 Model-View-Controller (MVC)

The project follows a Model-View-Controller (MVC) architecture to clearly separate responsibilities.

- **Model:** responsible for holding game-related data and logic, including entity modes/states, movement, positions and update functionality.
- **View:** responsible for the visual representation and drawing animations to the screen using an SFML Renderwindow and SFML functions to draw render and display these animations.
- **Controller:** the `World` class orchestrates the overall game logic and interactions between entities, such as collision detection or setting the different modes for an entity. Thus world acts like a coordinator of the game logic and is always a unique member of a level.

An important final remark is that the view layer does not contain any game logic, while the model and controller layers contain no SFML-specific code or rendering functionality. This strict separation ensures that the game logic can be compiled and used as a standalone library, fully independent of the chosen graphical representation.

3.2 Observer Pattern

The Observer pattern is used to update the view when the state of a model changes, without direct dependencies between the logic and representation layers. To prevent circular ownership and ensure memory safety, the **Subject** class (which serves as the base class for all entities) maintains a collection of `std::weak_ptr<Observer>`.

Each View is created and owned by the representation layer and attached to its corresponding Model as a `std::shared_ptr` through the concrete factory. The views observe their corresponding models without being owned by them, allowing the lifetime of visual components to be managed independently from the game logic.

In addition to entity views, the `Score` class also implements the Observer interface. It is owned by the game as a `std::shared_ptr` and attached to all relevant entities.

3.3 Abstract Factory Pattern

Entity creation is abstracted through an `AbstractFactory` interface, which is used by the `World` class in the logic library to create entities without any knowledge of their graphical representation.

A **ConcreteFactory** in the representation layer initiates the corresponding Model objects, attaches the appropriate SFML-based view classes, and returns a `std::unique_ptr` to the model. Ownership of all entities is then transferred to the **World**, ensuring a clear separation between game logic and representation.

3.4 State Pattern

The game flow, including the menu, gameplay, pausing, victory and game over is modeled using a Finite State Machine (FSM). A **State Stack**, managed by the **StateManager** and uniquely owned by the game, allows the current **LevelState** to be paused and resumed seamlessly through push and pop operations. Each state is responsible for deciding when a transition should occur, using a reference to the **StateManager**, ensuring that state transition logic remains encapsulated within the individual states.

3.5 Singleton

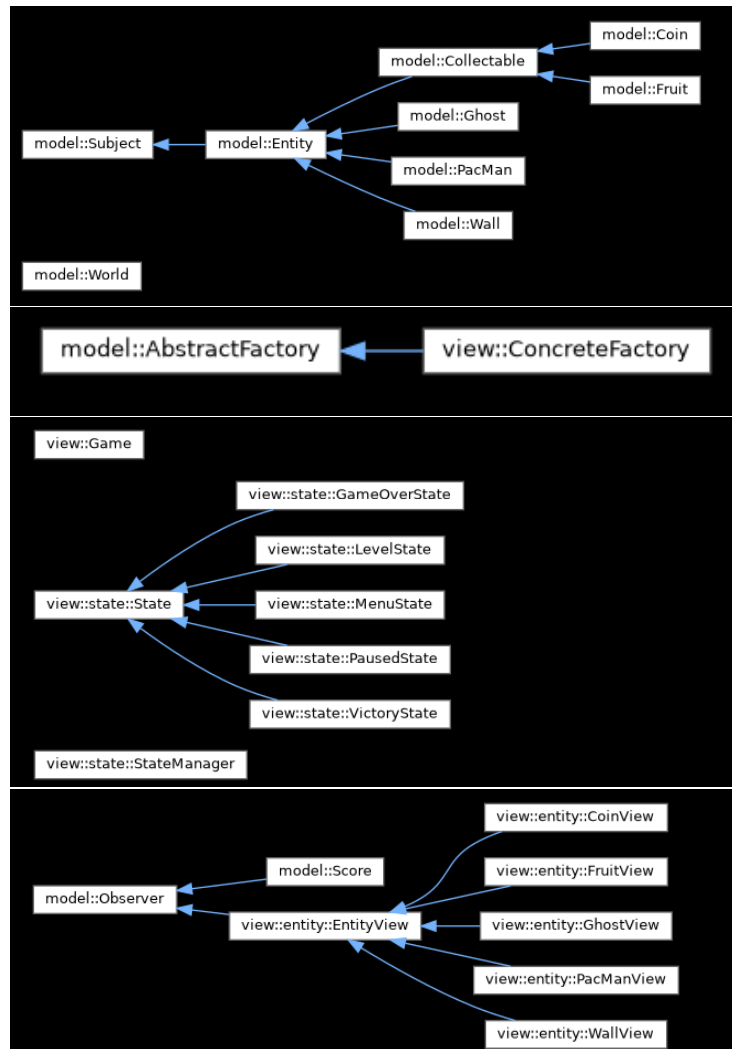
The **Clock** and **Random** classes are implemented as singletons, ensuring that exactly one global instance exists throughout the lifetime of the game. This is achieved by making their constructors private and deleting copy and assignment operators.

The **Clock** provides a consistent `deltaTime` value used for time-based updates, ensuring frame-rate independent game logic. The **Random** class encapsulates a seeded pseudo-random number generator, which is used for non-deterministic behavior such as breaking ties in the ghost AI.

4 Technical Implementation

- **Camera and Coordinates:** entity positions are project-modeled using a normalized coordinate system $([-1, 1])$, ensuring that the game logic remains independent of display resolution.
- **Smart Pointers:** the project exclusively uses smart pointers (`unique_ptr`, `shared_ptr`, and `weak_ptr`) to manage ownership and prevent memory leaks.
- **Ghost AI:** four unique AI behaviors are implemented, including Manhattan distance minimization for chasing and maximizing distance during fear mode.

5 Class Hierarchy



6 Additional Information

- **Defence video:** <https://youtu.be/52VKaC1NPug> (my apologies that the video is slightly longer than intended)
- **GitHub repository:** <https://github.com/VicoAerts/PacMan>